Python AI Engineer – Autonomous AI Agent

1. Introduction (30–45 sec)

"Hello everyone.

I'm Santosh Tambe, and today I'll demonstrate my Autonomous AI Agent built using Python, FastAPI, Google Gemini, and Streamlit.

The objective of this project is to accept a natural language request, automatically generate an execution plan, execute each task, perform reflection to improve the output, and finally generate a polished Microsoft Word document.

The project demonstrates autonomous planning, reasoning, execution, reflection, and document generation."

---

2. Architecture (2–3 min)

"Let's first understand the architecture.

The application consists of five major components.

The first component is the FastAPI backend, which exposes a REST endpoint POST /agent.

Whenever the user submits a request, FastAPI coordinates the complete workflow.

The second component is the Planner.

The planner sends the user request to Gemini and asks the model to generate an execution plan in JSON format.

Instead of hardcoding the workflow, the AI decides its own list of tasks.

The third component is the Executor.

The executor processes every task generated by the planner.

Each task is independently executed using Gemini, and the outputs are combined into one document.

The fourth component is the Reflection module.

After generating the document, another AI pass reviews the content.

Reflection checks grammar, formatting, consistency, clarity, and completeness before producing the final version.

The fifth component is the Document Generator.

Finally, the polished content is converted into a Microsoft Word document using python-docx.

The overall workflow is:

User Request

↓

Planner

↓

Execution Plan

↓

Executor

↓

Reflection

↓

Word Document

↓

API Response"

---

3. Technologies Used (45 sec)

"My project uses

- Python
- FastAPI for REST APIs
- Google Gemini 2.5 Flash as the LLM
- Streamlit for the user interface
- python-docx for document generation
- Pydantic for request validation
- Requests library for API communication."

---

4. Live Demo (Standard Request) (2 min)

"Now I'll demonstrate the first test case.

The request is:

Create a professional business proposal for an AI-powered hospital chatbot including executive summary, implementation strategy, budget, timeline, and conclusion.

After clicking Generate Document, the AI automatically performs several actions.

First, it understands the request.

Next, it generates its own execution plan.

Instead of following predefined logic, it decides the necessary tasks such as identifying objectives, defining features, estimating timeline, creating implementation strategy, and preparing conclusions.

Each task is executed independently.

The outputs are combined.

Reflection reviews the entire document.

Finally, the system generates a professional Microsoft Word document.

The API also returns

- execution plan
- final content
- document location
- status."

---

5. Live Demo (Complex Request) (2 min)

"The second example demonstrates autonomous decision making.

The request is:

Prepare meeting minutes for a product planning meeting discussing an AI customer support chatbot.

Assume missing participant information, identify discussion points, assign action items, estimate deadlines, and generate professional meeting minutes.

This request intentionally contains missing information.

Instead of failing, the planner makes reasonable assumptions.

It generates new tasks automatically.

The executor completes every section.

Reflection improves readability and consistency.

Finally, a complete Word document is generated."

---

6. Engineering Improvement (1 min)

"The engineering improvement I implemented is Reflection or Self-Check.

After the executor generates the complete document, the Reflection module performs another AI review.

It checks for

- grammatical errors
- duplicated information
- formatting consistency
- logical flow
- professional writing quality

This significantly improves document quality compared to returning the first generated output.

Reflection makes the agent more reliable and produces cleaner business documents."

---

7. Debugging Insight (1 min)

"During development I encountered an issue where the Streamlit interface continuously showed that the FastAPI server was not running.

The root cause was that the backend server was either not running or had stopped because of import and environment configuration issues.

To resolve this,

I verified the Python virtual environment,

installed all required dependencies,

validated imports,

started FastAPI independently using Uvicorn,

tested the API using the Swagger interface,

and only then connected the Streamlit frontend.

This systematic debugging approach isolated the problem quickly."

---

8. Engineering Tradeoff (1 min)

"The primary engineering tradeoff I made was choosing a Single-Agent architecture instead of a Multi-Agent architecture.

A multi-agent system could separate planning, execution, reflection, and validation into different autonomous agents.

However, considering the 60-minute development constraint, a single orchestrator controlling planner, executor, reflection, and document generation was much simpler to implement and easier to debug.

Although less extensible than a distributed multi-agent architecture, it provides lower complexity while still demonstrating autonomous planning and reasoning."

---

9. Conclusion (30 sec)

"In summary, this project demonstrates an end-to-end autonomous AI agent capable of

- understanding natural language,
- generating its own execution plan,
- executing tasks autonomously,
- reviewing its own work through reflection,
- generating polished Microsoft Word documents,
- and exposing the entire workflow through a FastAPI REST API.

Thank you for watching."